URL SHORTNER - SYSTEM DESIGN

1. Functional Requirements
2. Core Entities & API
3. Non Functional Requirements
4. Resource Estimation & Back of the Envelope Estimation
5. HLD
6. LLD (if possible)

1.Functional Requirements [Features required in the application]

- Example Use case:
    - Basic URL https://www.abc.com/q=chat&c=abc&d=efg&user=sjhdjshdsjdhjsdh%sdjshjhsj
    - Converted to TinyURL : https://wwwtinyURL.com/abc4545
    - If you click the TinyURL it will still redirect to the actual website.
    - Always generate a Unique URL
    - Short URL -> Long URL -> Website
    - Manage the link analytics => No of views for each link

2.Core Entities and API

Entities :

- Short URL
- Long URL
- UserID
- Dat Created
- Last Visited

API :

- Shorten URL : POST api/urls/shorten
    - Req : LongURL
    - Res : shortURL
- Go to Website : GET api/urls/{shortURL}
    - Req : shortURL
    - Res : redirected to Long URL => Status 302

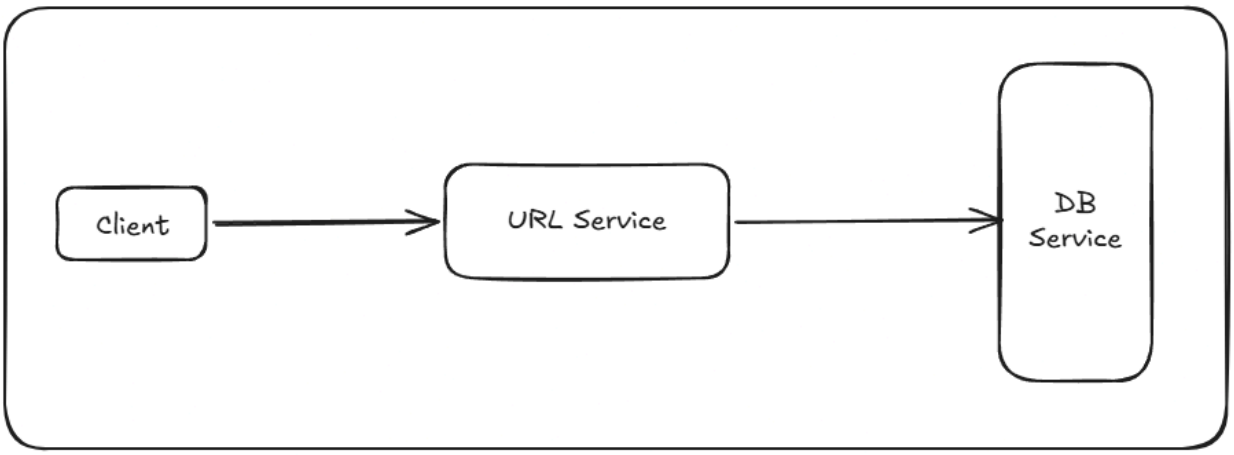
3. Non Functional Requirements

- Scalability : Able to reach a large number of users with no down time
- Availability : Application availability for Processing at all times

- Low Latency : Reduced processing time
- Availability & Partition Tolerance Tradeoff VS Consistency
    - The URL's (shortURL) need to be high available.
    - The URL can be eventually updated across the DB's.
    - So eventually consistency is possible.
- Low length of URL (len < 7)

4. BOTE & Resource Estimation
    - Consider Total no: of User's : 10M (10^6)
    - Total URLS per User: 20 per day
    - Total URLS generated : 10*(10^6) * 20 = 2B
    - 2B req / day => 2*(10^10) / 864000= 20K req per sec

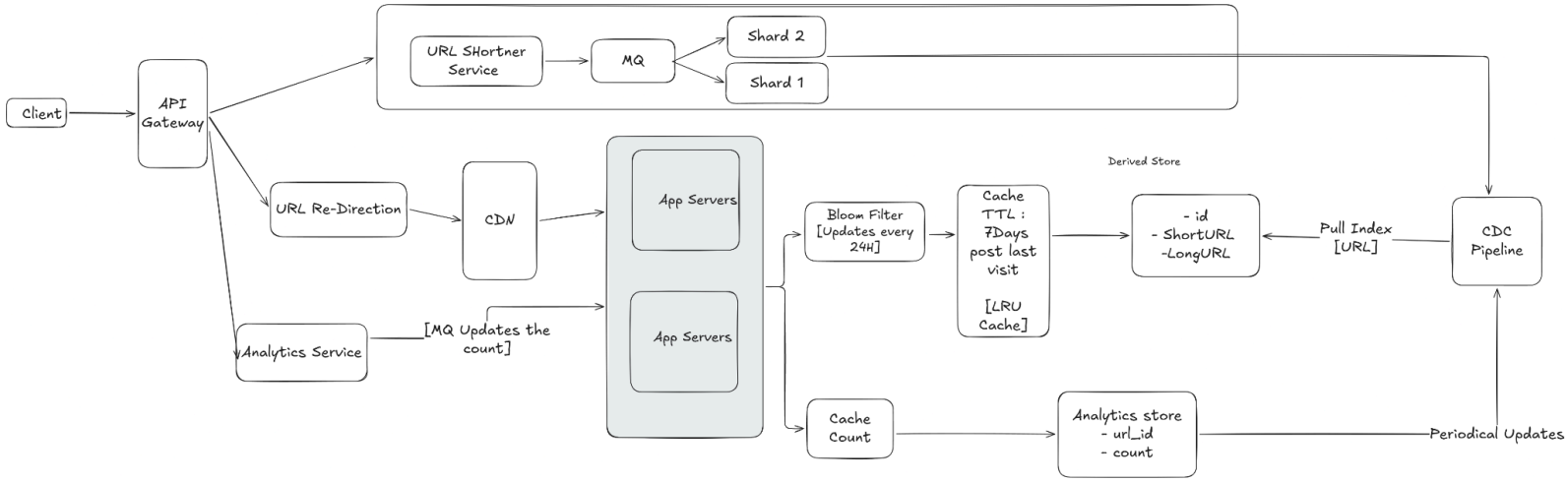
5. High Level Design
    - Initial Design



This design ensures we get proper NFR like
- Availability
- Consistency

However there might be issues in terms of
- Traffic management
- Services scale
- No caching
- Easier for analytics
- Difficult to manage in a monolithic structure.

- Updated Diagram

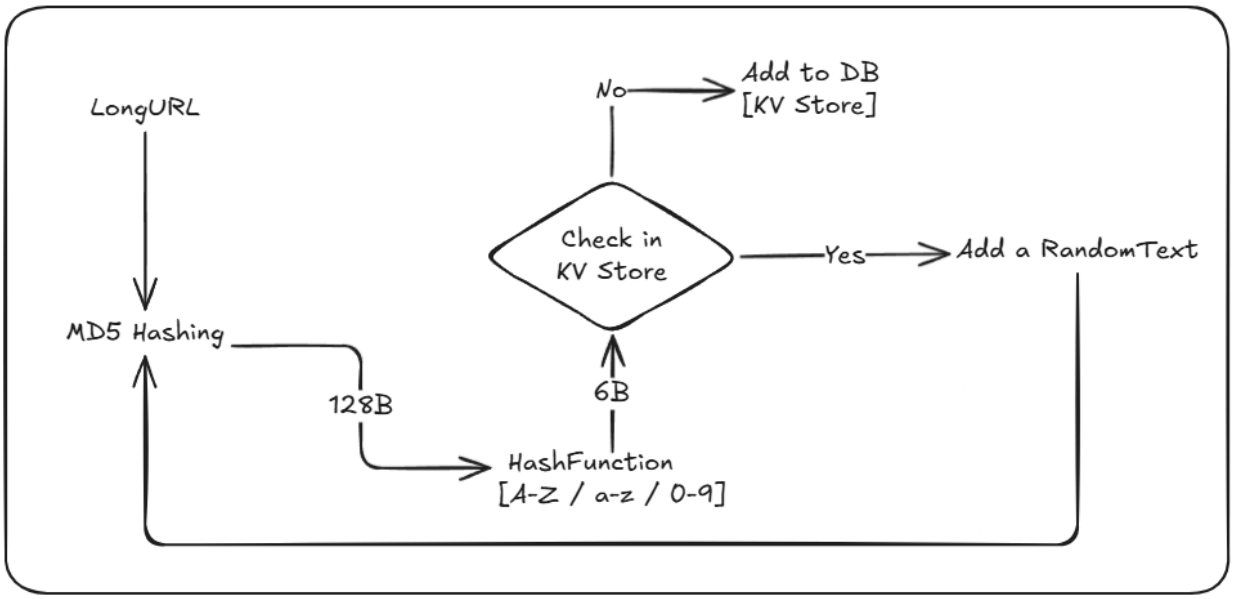


- An API Gateway which is going to be a central source of truth to direct the user request to the server which can handle it.
- Since there are two main processes which is a read and write of the long URL, we are going to go towards a microservice architecture where we separate the load of a single server into multiple services.
- The Read servers will have more compute power for faster, and Hot URL Caching.
- The write services will have a message queue to send and process the user's requests.
- The data will be stored in two shards based on availability.
- The data updated in the shard will be periodically updated in the nearest Derived Store using the Change Data Capture.
- The Nearest Derived data store pulls the updated index.
- Upon user request for redirection to the main URL using the short URL, the request goes through two processes
    - ReDirect the user
        - A global CDN where the longURL is cached and the user is shown with a system status code 302 (Temporarily moved).
        - We are using the 302 Code here as we need to track the count of traffic/users visiting it.301 will cache the URL by default in edge.
        - If there is a CDN miss it foes to the application servers where the read process occurs.
        - This request is processed to the bloom filter to check if the shortURL exists or not
        - Once exists the Long URL is fetched from the nearest cache.
        - The cache shall be using a LRU Algorithm.
    - Store the URL Analytics
        - Store the views for the URL in a message queue.
        - This count will eventually be updated in the nearest Redis.
        - Store in the nearest Analytics store
        - Eventually Update in the DB.

Deletion Policy in Cache & DB.

- The cache will be using a LRU Algorithm which will be used for eviction of the data from the cache.
- The data in the DB will be deleted in batches based on the last 7 days of activity.
    - If the URL is not used for >7 days, then the url is going to be deleted.
    - The Data which is deleted in the DB, will also be reflected in the Derived store.
    - The Bloom filter will also be updated in near real time.
    - The Cache CDN which holds the URL's will also be updated on every DB Deletion.

6. LLD -> Implementation of Hashing on the URL



The Implementation of the URL shortener is done by using two main algorithm :

- MD5
- Base 62 Encoding

Process :

- The MD5 Hashing algorithm converts the length of the URL into 22 character of length.
- This encrypted string is now converted into base62 value which will be of A-Z, a-z, 0-9.
- The maximum length of the URL can be 6, with each place having an option of 62 char.

- So total choices of URLS : 62 ^ 6 : 6B .
- If the URL already exists we can add in a random character into the long URL and perform the entire operation again.
- Once the URL is unique, it is added to the DB.
- However this would lead to
    - High Collision
    - URL Uniqueness cannot be possible.